

C++ - IHM avec Qt

BTS CIEL



Présentation de Qt

Qt est un framework C++ cross-platform qui s'appuie fortement sur la programmation orienté objet.

- **Modules importants :**
 - **Qt Widgets** : Interfaces classiques (boutons, fenêtres, etc.).
 - **Qt Quick** : Interfaces modernes (QML/JavaScript).
 - **Qt Core** : Conteneurs, threads, fichiers, etc.
 - **Qt Network** : Réseau (TCP, HTTP, WebSocket).
 - **Qt SerialBus** : Communication CAN, Modbus.

Présentation de Qt



NOKIA



Qt Group



Présentation de Qt



Présentation de Qt



Cross-platform ?



Qt Widget VS Qt Quick

Critère	Qt Widgets	Qt Quick
Langage	C++ (POO)	QML / C++
Style d'interface	Classique (bureautique)	Moderne et fluide (animations)
Rendu graphique	Logiciel (CPU)	Accéléré (GPU)
Cible embarquée	Économe en ressources	Riche en fonctionnalités

Qt Widget VS Qt Quick

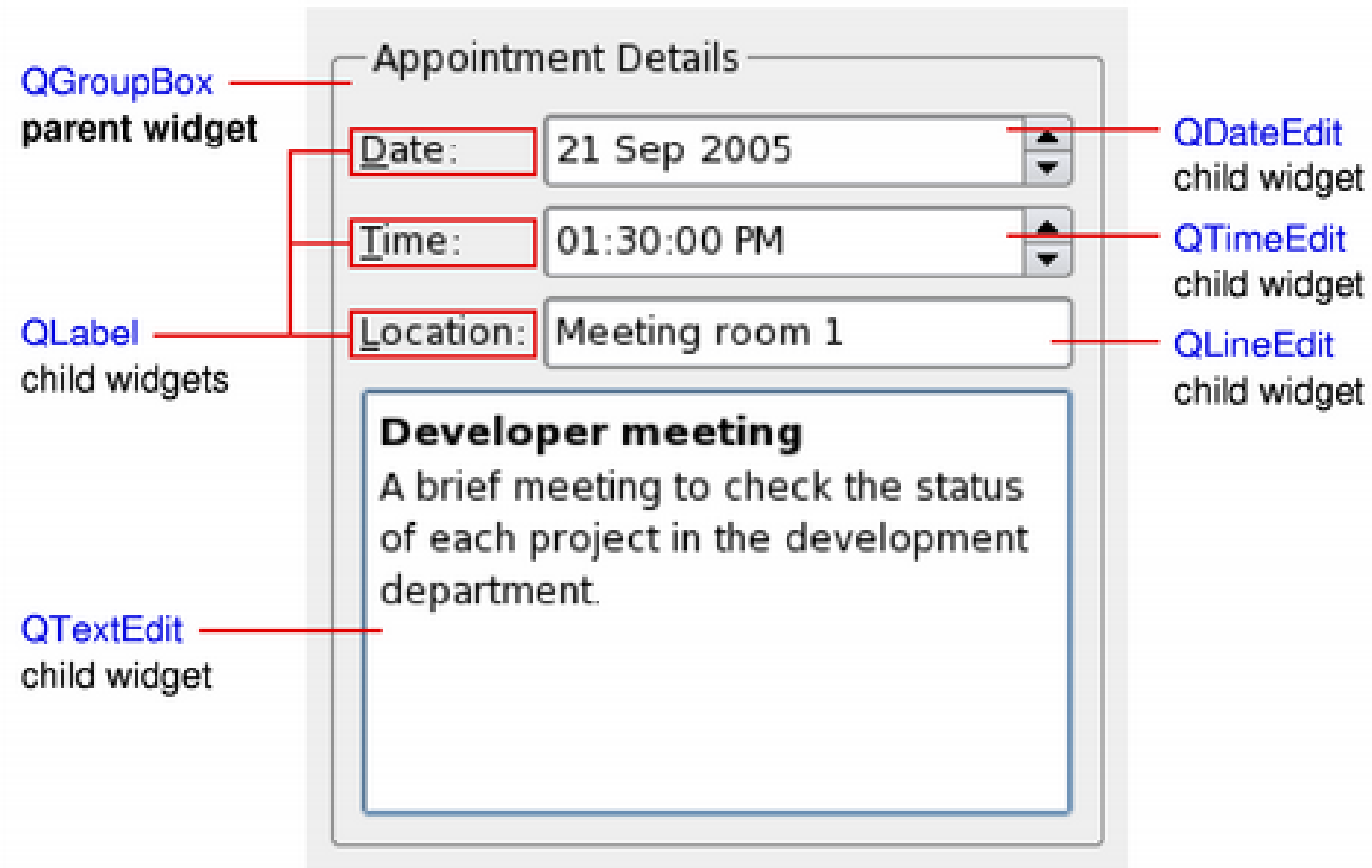
Exemple de programme QML

```
import QtQuick

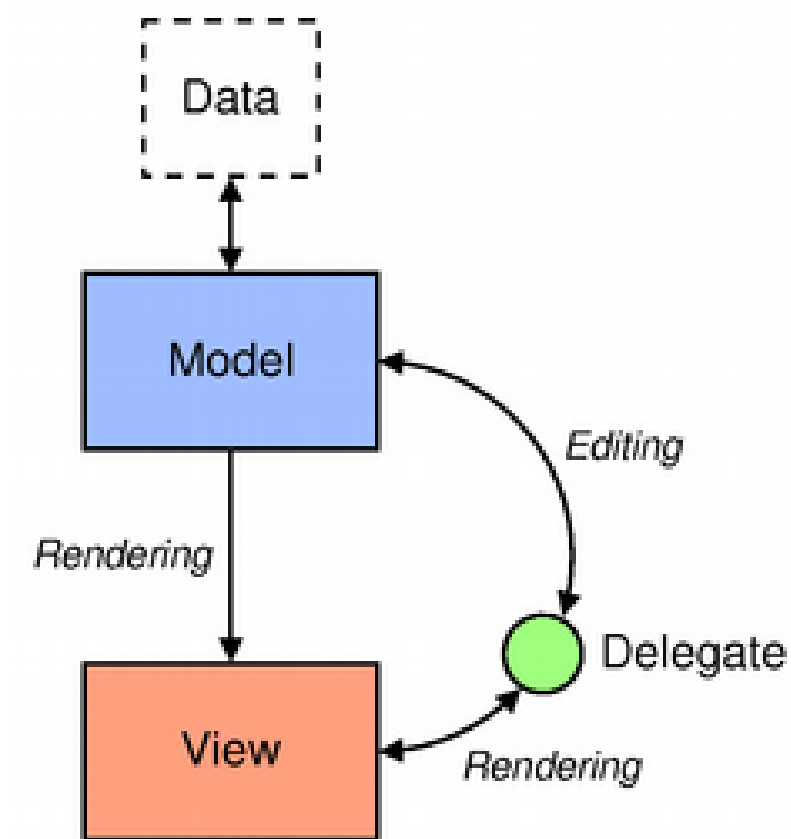
Rectangle {
    id: canvas
    width: 250
    height: 200
    color: "blue"

    Image {
        id: logo
        source: "pics/logo.png"
        anchors.centerIn: parent
        x: canvas.height / 5
    }
}
```


Qt Widget



Architecture Vue-Modèle



Architecture Vue-Modèle

Modèle

Le modèle prend la forme d'une **instance de classe**.

Cette classe doit **hériter** de `QAbstractItemModel` ou de l'un de ses héritiers :

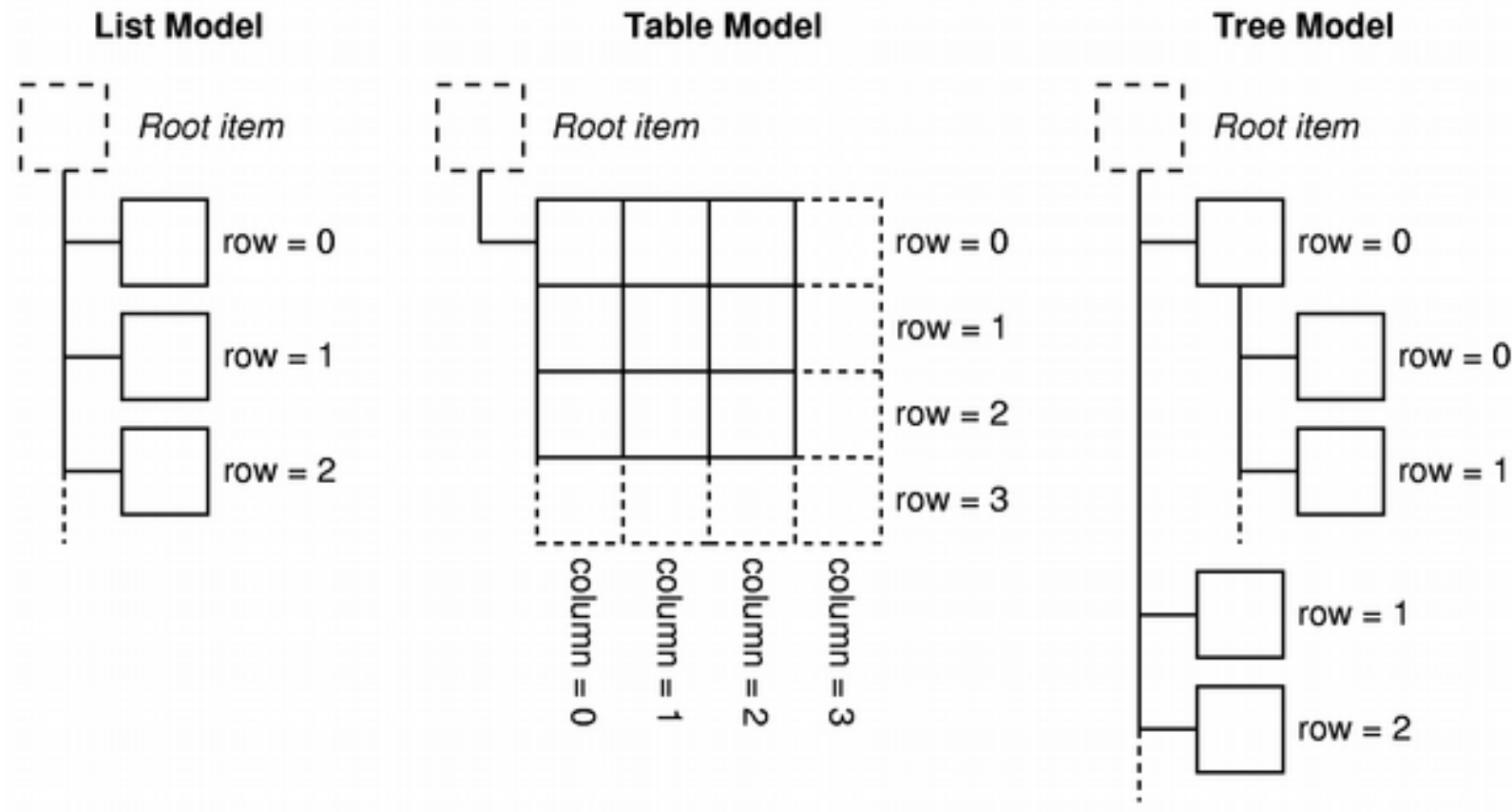
- `QAbstractListModel` pour les ensembles **unidimensionnels** (une liste)
- `QAbstractTableModel` pour les ensembles **multidimensionnels** (un tableau)

Qt offre des modèles plus avancés pour travailler avec des sources de données standards :

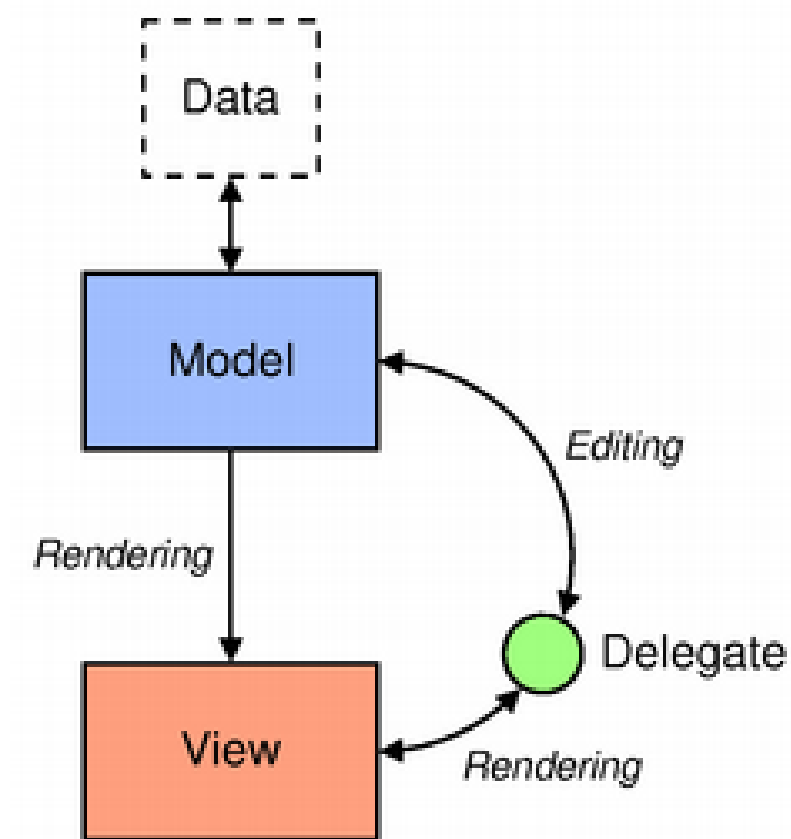
- `QStringListModel` pour stocker une liste de chaînes de caractères
- `QFileSystemModel` pour stocker des informations du système de stockage local (arborescence)
- `QSqlQueryModel` pour l'accès aux bases de données relationnelles (SQL)

Architecture Vue-Modèle

Modèle



Architecture Vue-Modèle



Architecture Vue-Modèle

Vue

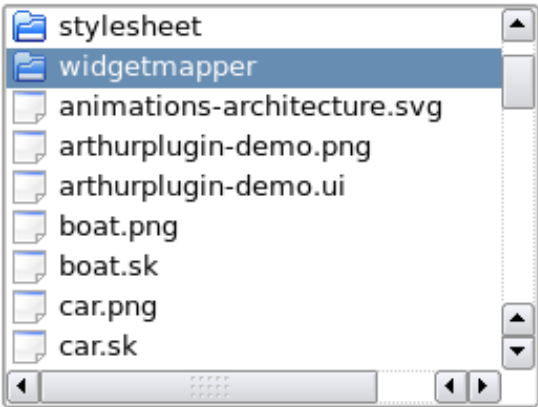
La vue prend la forme d'une **instance de classe**.

Cette classe doit hériter de `QAbstractItemView` ou de l'un de ses héritiers :

- `QListView` pour représenter les données sous forme de liste
- `QTableView` pour représenter les données sous forme de tableau
- `QTreeView` pour représenter les données sous forme d'arbre

Architecture Vue-Modèle

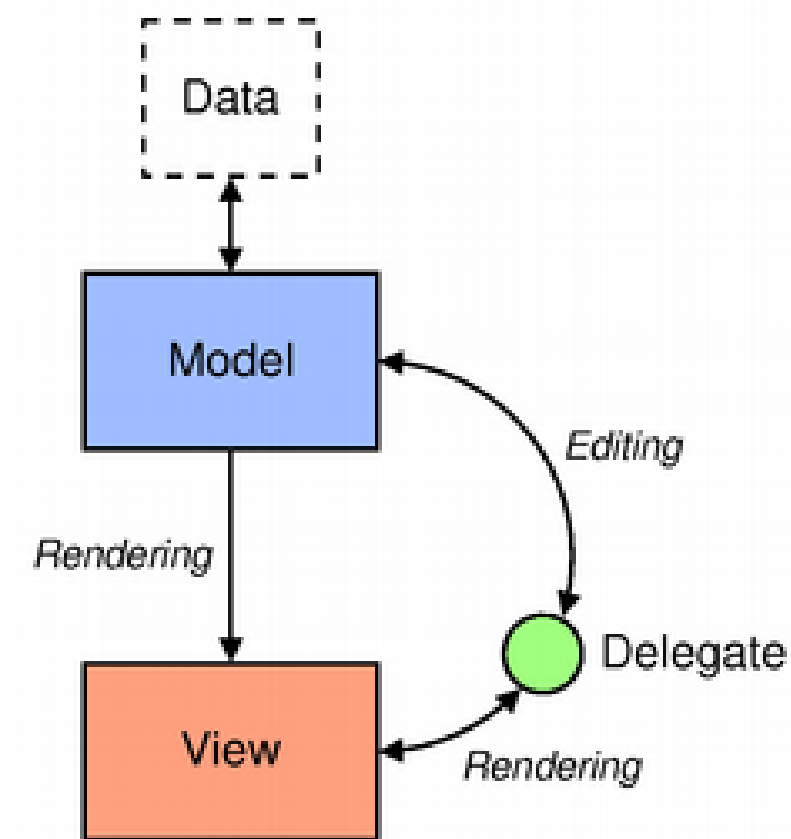
Vue



Name	Size	Type	Date
qtopiaco		Folder	9/25
stylesheet		Folder	4/23
widgetmapper		Folder	4/23
sql-widget-map...	11 KB	png File	4/23
widgetmapper-...	3 KB	sk File	4/23
animations-archi...	14 KB	svg File	9/25
arthurplugin-demo...	58 KB	png File	4/23
arthurplugin-demo.ui	1 KB	ui File	4/23

	Name	Size
12	widgetmapper	
13	animations-archi...	
14	arthurplugin-dem...	
15	arthurplugin-dem...	
16	boat.png	

Architecture Vue-Modèle



Architecture Vue-Modèle

Délégué (delegate)

Le délégué prend la forme d'une **instance de classe**.

Cette classe doit hériter de `QAbstractItemDelegate`.

La classe généralement utilisée est `QStyledItemDelegate`.

Architecture Vue-Modèle

Exemple complet

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    QSplitter *splitter = new QSplitter;

    // Model
    QFileSystemModel *model = new QFileSystemModel;
    model->setRootPath(QDir::currentPath());

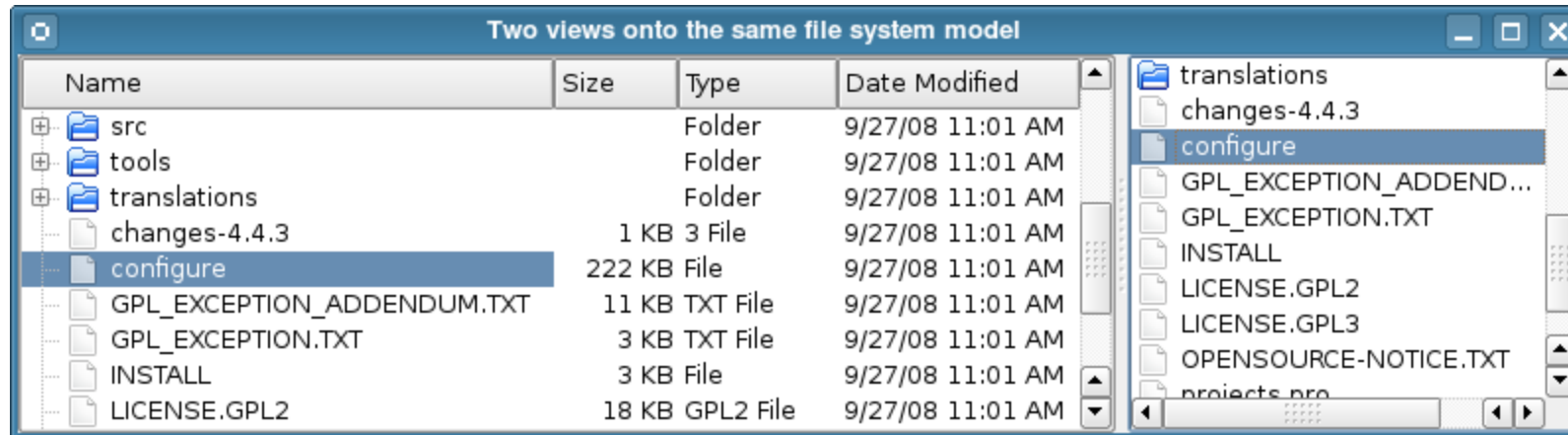
    // View
    QTreeView *tree = new QTreeView(splitter);
    tree->setModel(model);
    tree->setRootIndex(model->index(QDir::currentPath()));

    QListView *list = new QListView(splitter);
    list->setModel(model);
    list->setRootIndex(model->index(QDir::currentPath()));

    splitter->setWindowTitle("Two views onto the same file system model");
    splitter->show();
    return app.exec();
}
```

Architecture Vue-Modèle

Exemple complet



Architecture Vue-Modèle

Signals et slots (programmation événementielle)



Architecture Vue-Modèle

Signals et slots

```
#include <QApplication>
#include <QPushButton>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QPushButton bouton("Quitter");

    QObject::connect(&bouton, &QPushButton::clicked, &app, &QApplication::quit);

    bouton.show();

    return app.exec();
}
```

Architecture Vue-Modèle

Signals et slots

```
#include <QApplication>
#include <QPushButton>
#include <iostream>

void onClick()
{
    std::cout << "Bouton cliqué !" << std::endl;
}

int main(int argc, char *argv[])
{
    // ...
    QObject::connect(&bouton, &QPushButton::clicked, &onClick);
    // ...
}
```

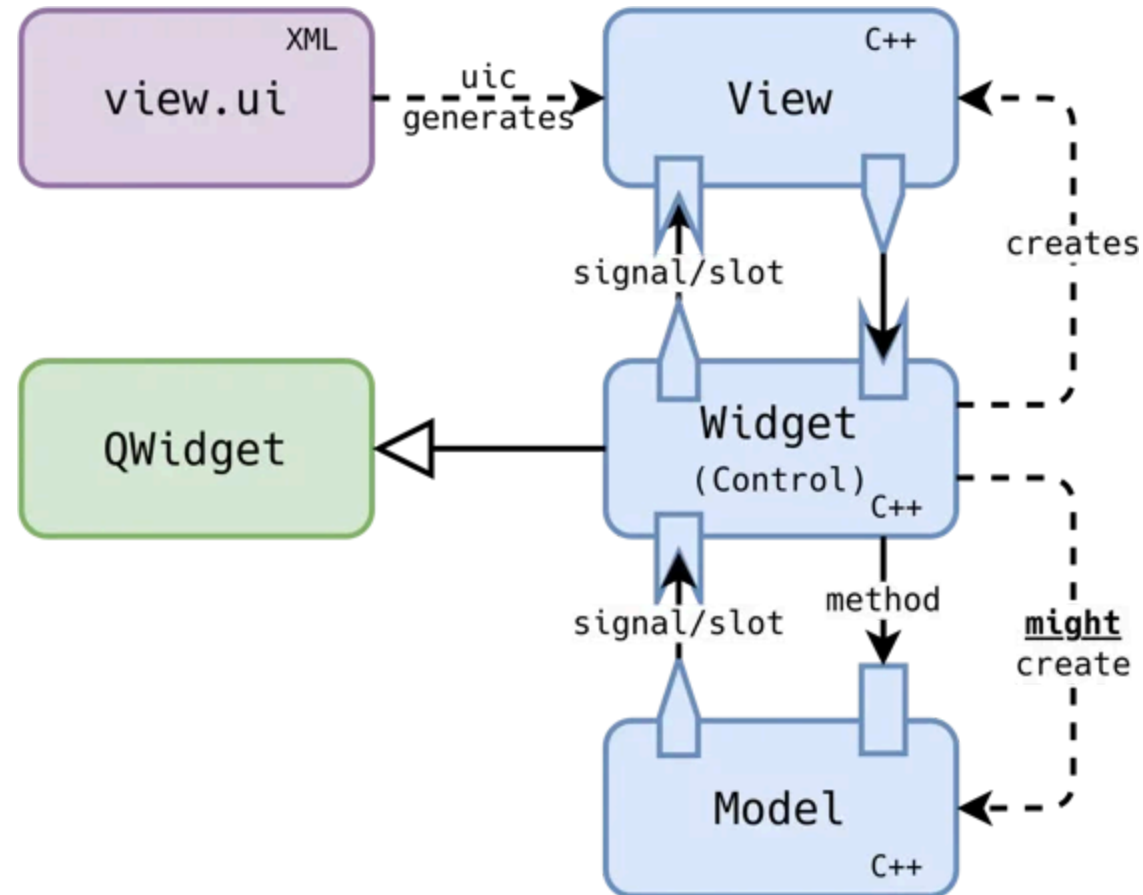
Architecture Vue-Modèle

Signals et slots

- Les signaux émis par **le modèle** informant la vue **des modifications** apportées aux données détenues par la source de données.
- Les signaux émis par **la vue** fournissent des informations sur **les interactions** de l'utilisateur avec les éléments affichés.
- Les signaux émis par **le délégué** sont utilisés pendant l'édition pour informer le modèle et la vue de **l'état de l'éditeur**.

Architecture Vue-Modèle

Notion de widget



Gestion de la mémoire dans Qt

Il n'y a pas un problème ?

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    QSplitter *splitter = new QSplitter;

    // Model
    QFileSystemModel *model = new QFileSystemModel;
    model->setRootPath(QDir::currentPath());

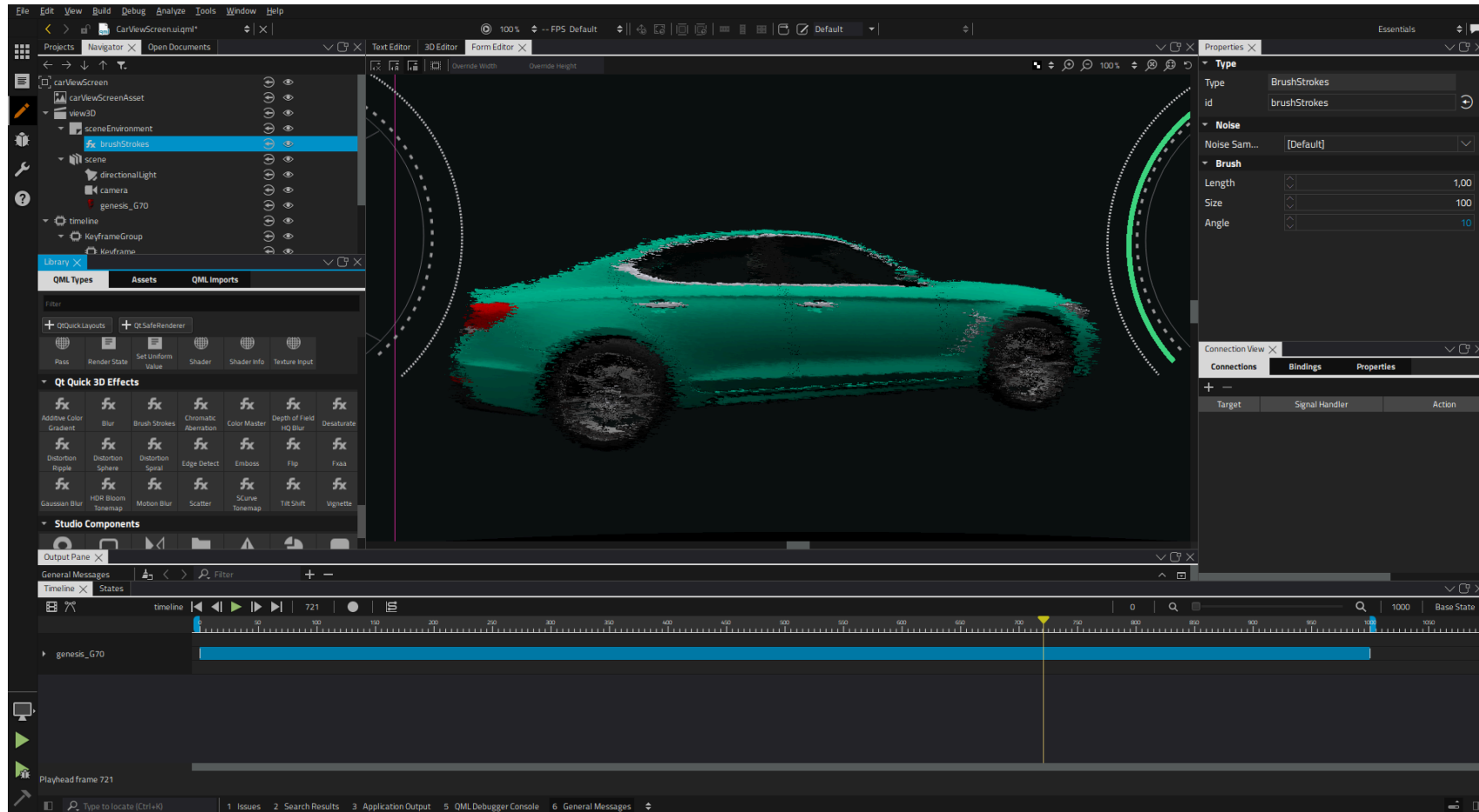
    // View
    QTreeView *tree = new QTreeView(splitter);
    tree->setModel(model);
    tree->setRootIndex(model->index(QDir::currentPath()));

    QListView *list = new QListView(splitter);
    list->setModel(model);
    list->setRootIndex(model->index(QDir::currentPath()));

    splitter->setWindowTitle("Two views onto the same file system model");
    splitter->show();
    return app.exec();
}
```

Les outils

Qt Creator



Les licences Qt

Add-Ons (GPLv3+ or LGPLv3)

Canvas 3D	Charts	Qt Quick 2D renderer	Data Visualization	Purchasing
Active Qt	Graphical Effects	NFC	Location	Qt 3D
X11, Windows, Mac Extras	Print Support	Sensors	Concurrent	WebEngine
Android Extras	Image Formats	Positioning	Serial Port	WebSockets
XML & XML Patterns	SVG	Bluetooth	D-Bus	WebChannel

Essentials (GPLv2+/LGPLv3)

		Multimedia Widgets	Quick Dialogs	Quick Controls
GUI	Widgets	Multimedia	Quick Layouts	Quick
Core	Network	SQL	Test	QML

Desktop & mobile platforms (GPLv2+/LGPLv3)

Windows	Mac	Linux Desktop	Android	iOS	WinRT
---------	-----	---------------	---------	-----	-------

Development Tools (GPLv3)

Qt Creator Cross-platform IDE	CPU usage analyzer
Qt Designer GUI Designer	GPU Profiler
Qt Linquist I18N Toolset	Clang static analyzer
Qt Assistant Documentation Tool	Qt Quick Compiler
Moc, uic, rcc Build Tools	Qt Quick Profiler
Qmake Cross-platform Build Tool	Autotest integration

License Legend

LGPLv2.1
LGPLv3
GPLv3
GPLv3, new modules for OSS
Commercial, unavailable for OSS